Supervised Learning

What is Machine Learning?

Machine learning “gives computers the ability to learn without being explicitly programmed”

- Arthur Samuel

The goal of machine learning is generalization: to learn a general rule from many specific examples (sample data from a distribution) that works well when applied to new examples (out-of-sample but still in distribution data).

Supervised Learning (a.k.a. Function approximation)

- Learning from labeled tabular data
- This labeled data is called training data, and consists of observations and corresponding labels (*a.k.a.* ground truth).
- The goal of a supervised learning algorithm is to approximate a function that generalizes well to unseen examples.
- Supervised algorithms are typically evaluated on validation data, and then deployed on test data.

Supervised Learning

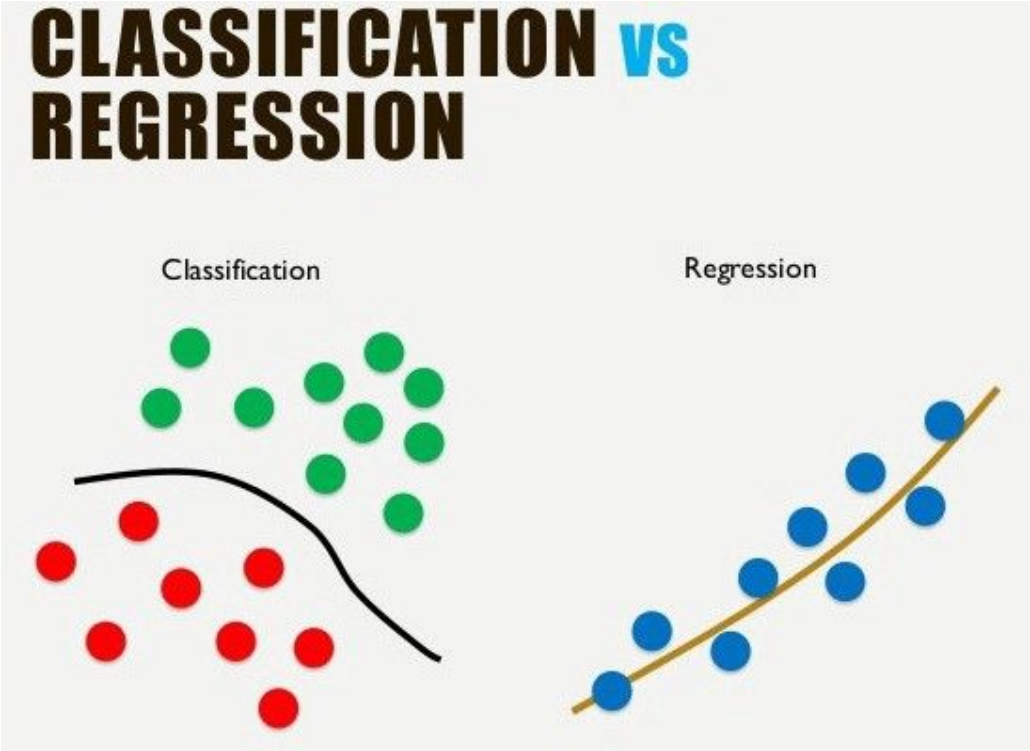


Image Source

k Nearest Neighbors

k Nearest Neighbors

To classify an observation:

- Look at the labels of some number, say *k*, of neighboring observations.
- The observation is then classified based on its nearest neighbors' labels, using, for example, majority vote.

Instance-based learning

- … as opposed to model-based
- There is no model: i.e., no learning phase



Image Source

k-NN caveats

- *k*-NN can be very slow, especially for very large data sets
  - *k*-NN is not a learning algorithm in the traditional sense, because it doesn't actually do any learning: i.e., it doesn't preprocess the data
  - Instead, when it is given a new observation, it calculates the distance between that observation and every existing observation in the data set
- *k*-NN works better with quantitative data than categorical data
  - Data must be quantitative to calculate distances
  - So categorical data must be transformed
- Without clusters in the training data, *k*-NN cannot work well

k Nearest Neighbors

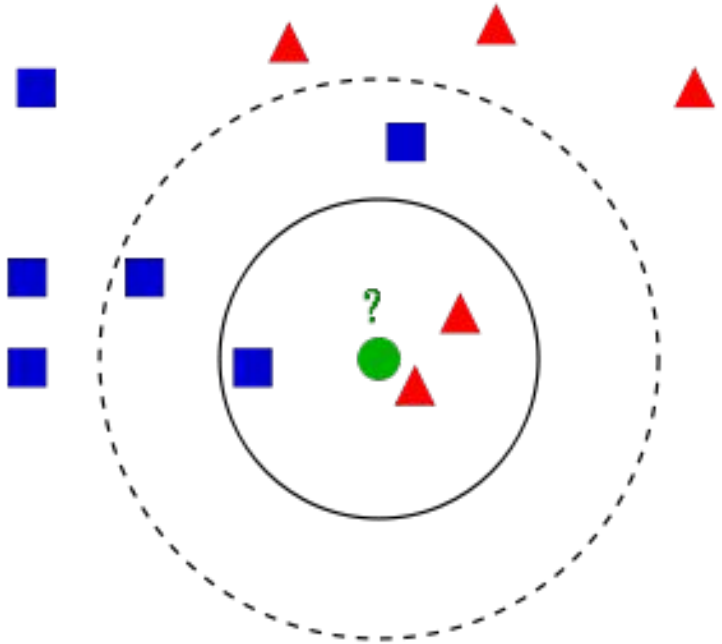


Image Source

Example

- Let’s try to classify the unknown green point by looking at *k* = 3 and *k* = nearest neighbors
- For *k* = 3, we see 2 triangles and 1 square; so we might classify the point as a triangle
- For *k* = 5, we see 2 triangles and 3 squares; so we might classify the point as a square
- Typically, we classify by some variant of majority vote, so use an odd value of *k* to avoid ties

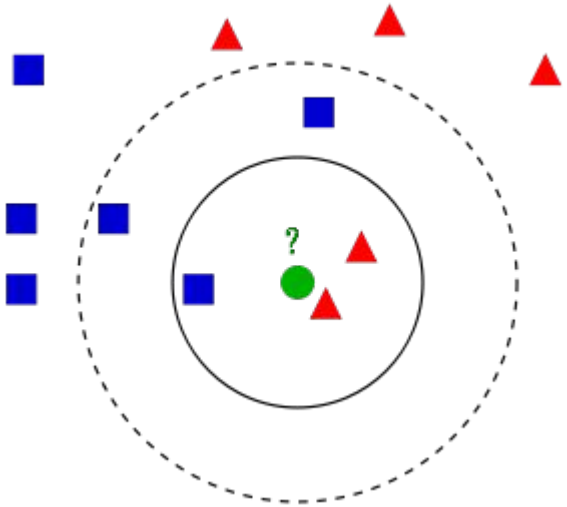
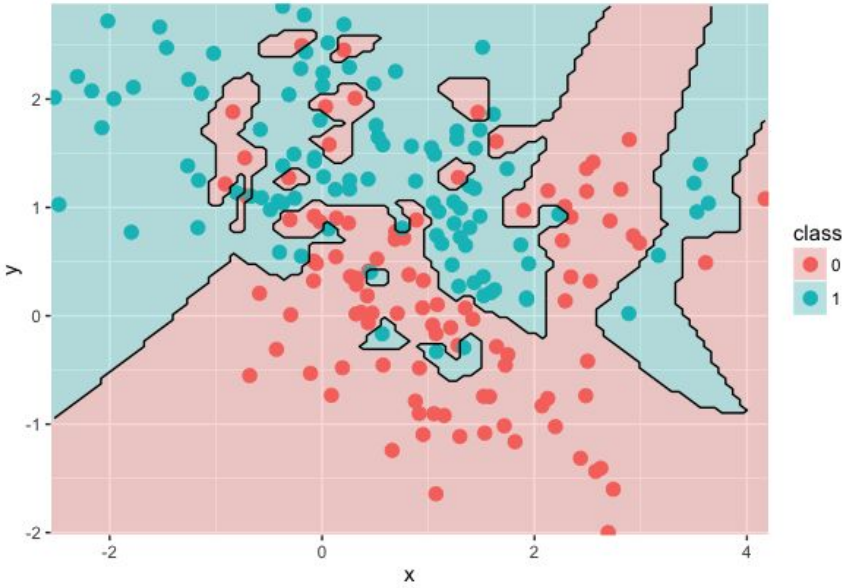


Image Source

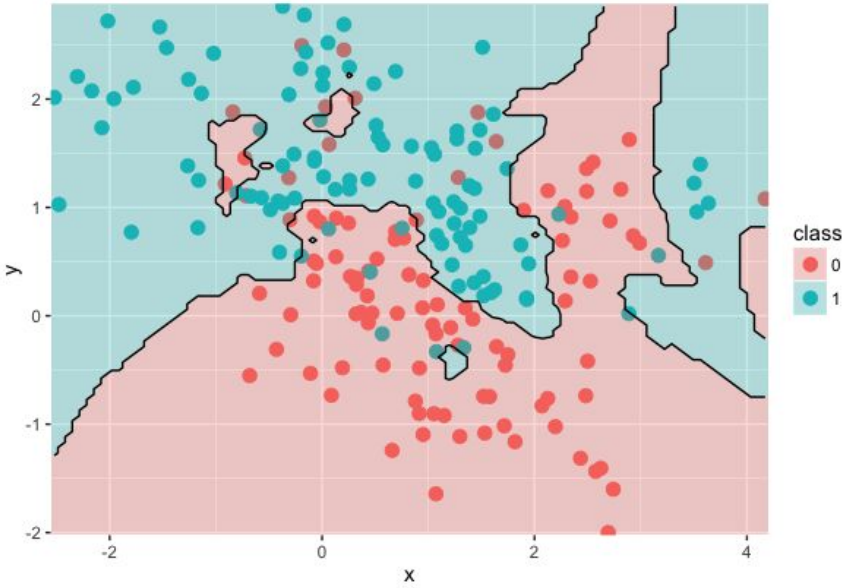
Key Design Decisions

- Choose *k*
  - Choose a threshold
- Define “neighbor”
  - Define a measure of distance/closeness
    (Make sure measurement values are comparable)
- Decide how to classify based on neighbors’ labels
  - By a majority vote, or
  - By a weighted majority vote (weighted by distance), or …

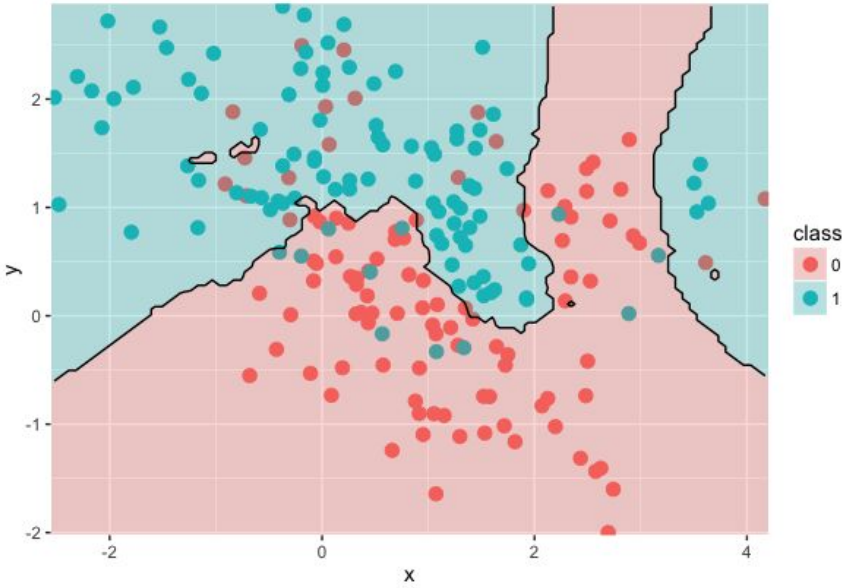
$k = 1$



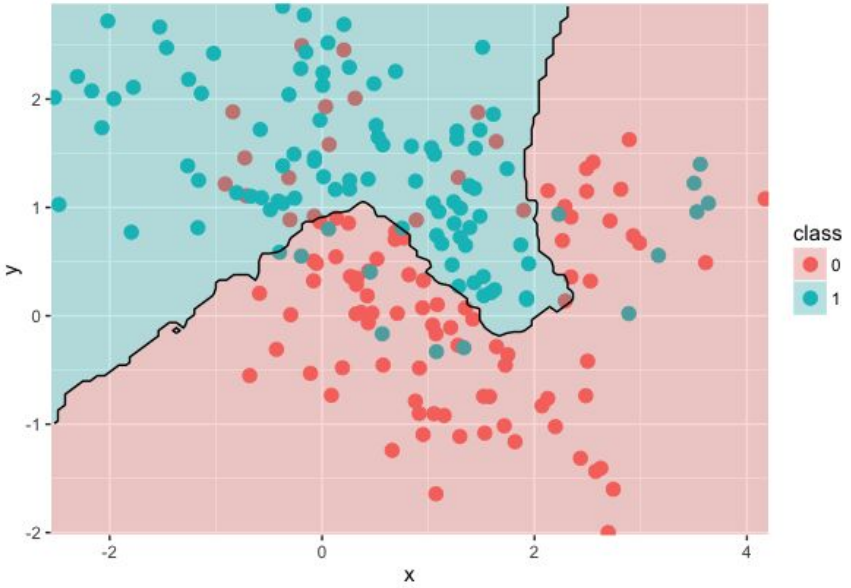
$k = 3$



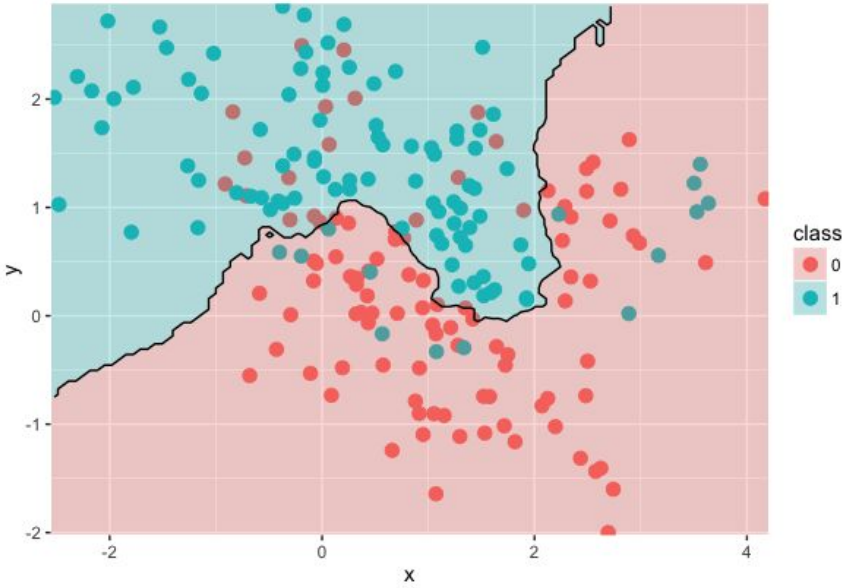
$k = 7$



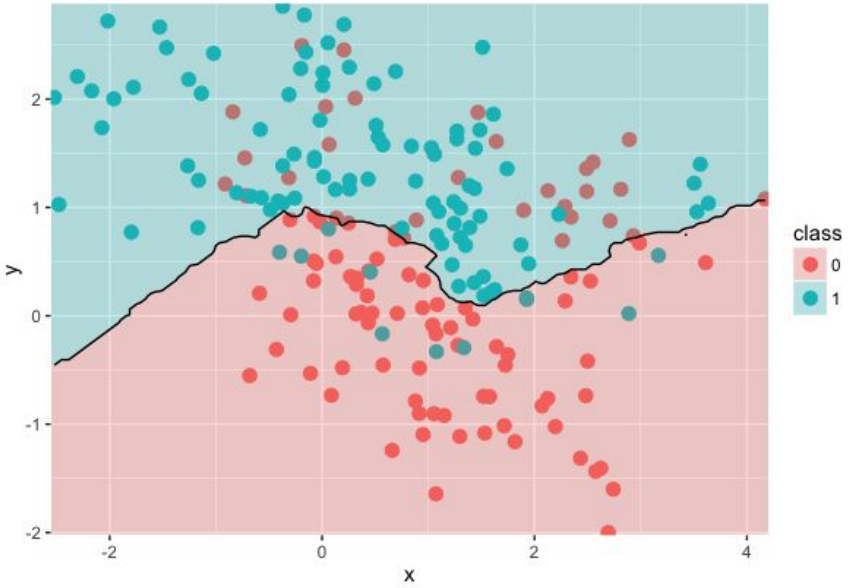
$k = 15$



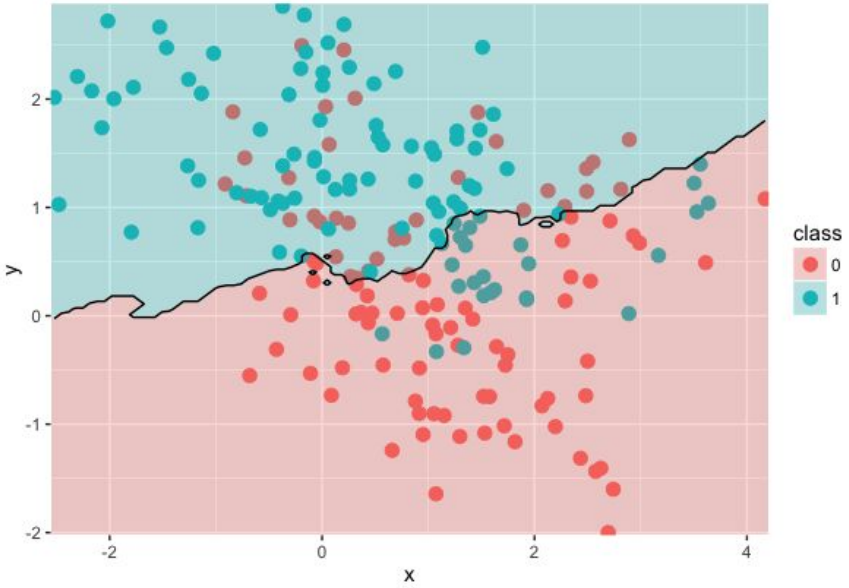
$k = 25$



$k = 51$



$k = 101$



Model Selection

Model Selection

Box: “All models are wrong, but some are useful.”

Occam’s Razor: The simplest explanation is best!

Einstein: “Everything should be made as simple as possible, but no simpler.”

Overfitting

- Problem: Models are always biased towards training data
- A model overfits when it “memorizes” the training data
- Overfit models cannot generalize well to test data
- Solution: Hold out data to evaluate models to mitigate the risk that they overfit

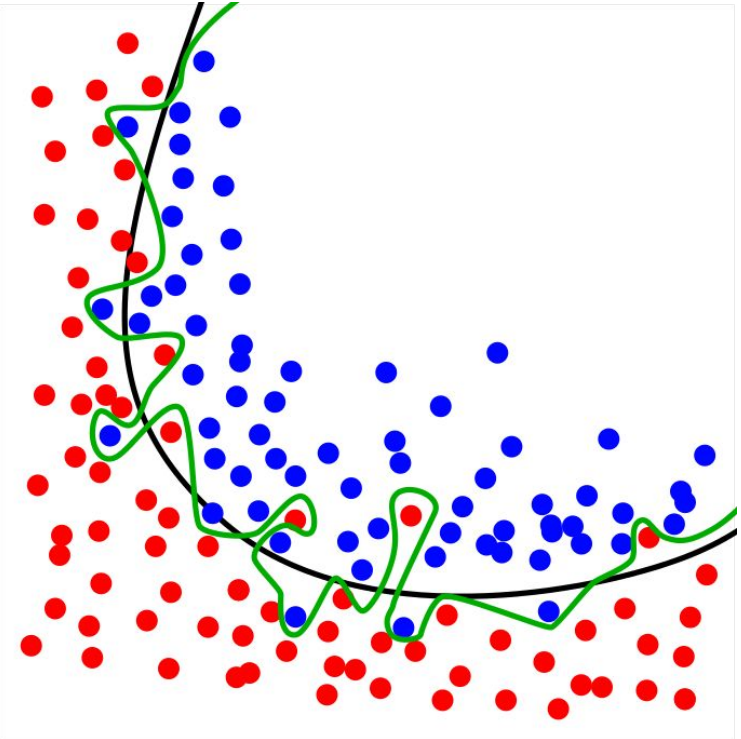


Image Source

Balancing Overfitting and Underfitting

- Overfitting: model varies too much with the data
  - Make few to no structural assumptions
  - Throw in everything in the kitchen sink!
  - Low bias, high variance
- Underfitting: model doesn't vary enough with the data
  - Insist on a simple relationship among variables (e.g., linear)
  - Select only a few key features
  - High bias, low variance

Goal: find a model that appropriately generalizes, by balancing complexity (overfitting) and simplicity (underfitting): i.e., by trading off bias for variance

The Bias-Variance Decomposition Theorem

- Estimator $\hat{\theta}$: rule for estimating an unknown population parameter from data
  - E.g., the sample mean is an estimator
- Bias: difference between the expected value of an estimator and the true value of the population parameter: $E[\hat{\theta}] - \theta$
- Variance of an estimator measures how much the estimates it produces vary from one sample to another: $E[(\hat{\theta} - E[\hat{\theta}])^2]$

$$\text{Expected Model Error} = E[(\hat{\theta} - \theta)^2] = \text{Bias}^2(\hat{\theta}) + \text{Variance}(\hat{\theta})$$

The Bias-Variance Decomposition Theorem

- Estimator $\hat{\theta}$: rule for estimating an unknown population parameter from data
  - E.g., the sample mean is an estimator
- Bias: difference between the expected value of an estimator and the true value of the population parameter: $E[\hat{\theta}] - \theta$
- Variance of an estimator measures how much the estimates it produces vary from one sample to another: $E[(\hat{\theta} - E[\hat{\theta}])^2]$
- Irreducible Error: error in even a perfect model (due to unmeasured factors, measurement error, inherent randomness, etc.)

$$\text{Expected Model Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

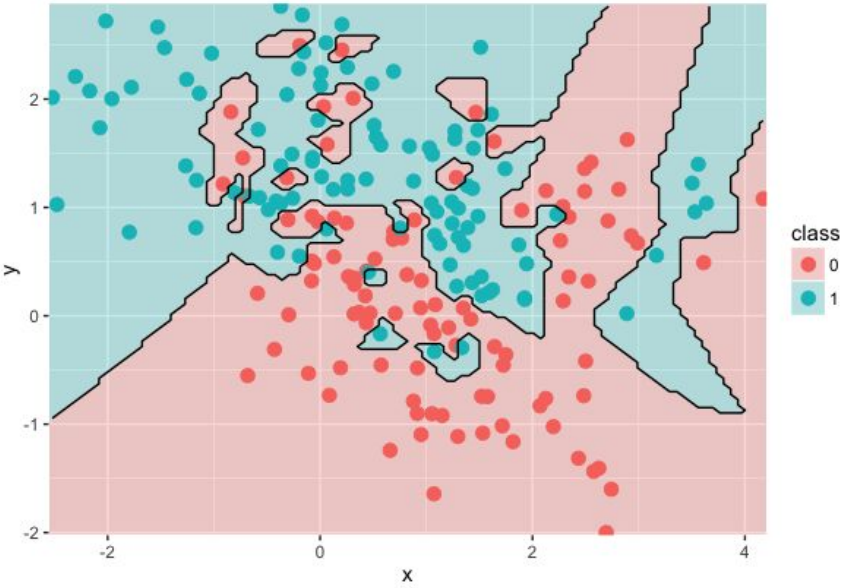
*k*NN Model Selection

Choose a *k* that appropriately balances complexity and generalization capabilities: i.e., that optimizes the bias-variance tradeoff.

- Low bias, high variance
  - With low values of *k*, the very jagged decision boundaries are a sign of high variance
- High bias, low variance
  - A high value of *k* indicates a high degree of bias, but contains the variance

Small *k*

- *k* = 1
- Low bias
- Models like this are overfit
- Reading too much into training data, extrapolating based on things that aren't really there
- High variance

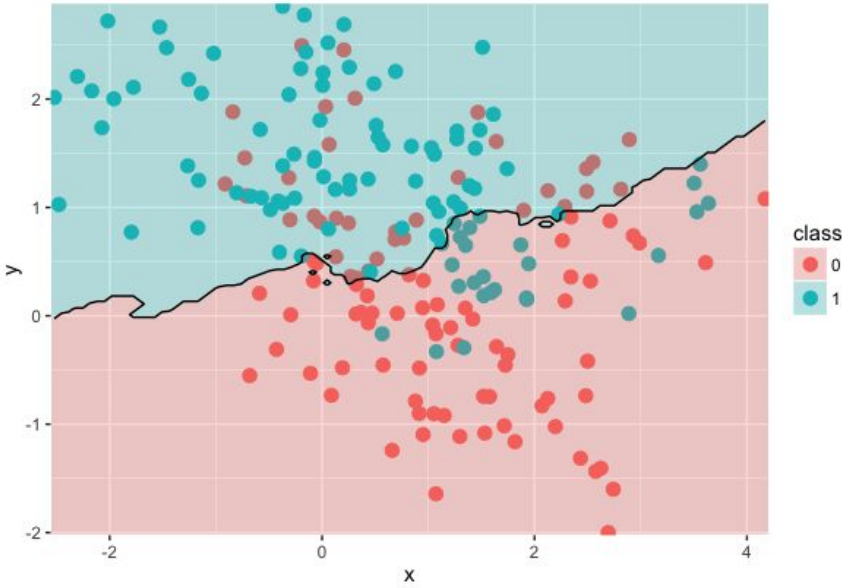


k = 1: Low bias, High Variance

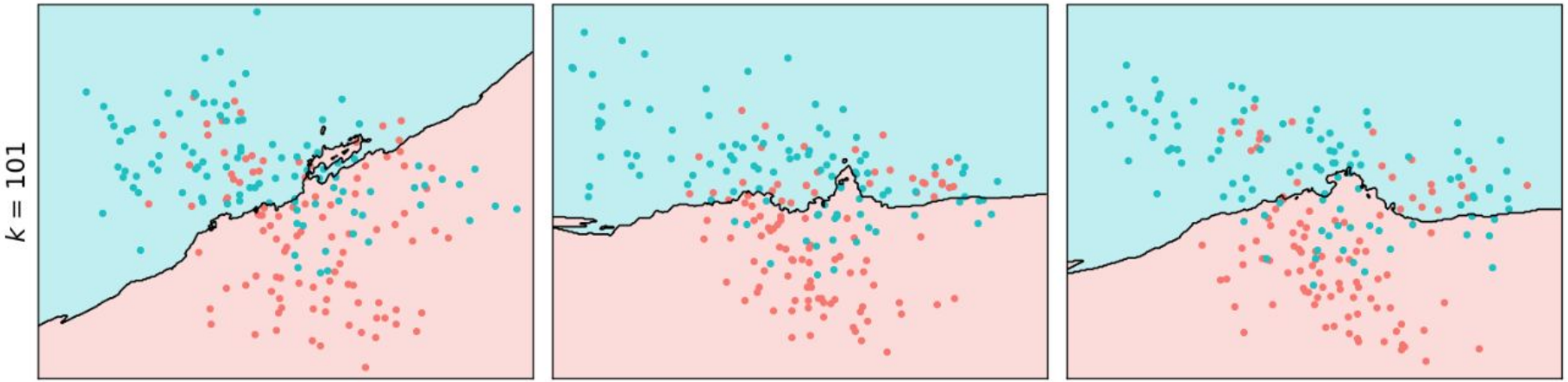


Large *k*

- *k* = 101
- High bias
- Rather than being overfit, this model is underfit
- The decision boundary doesn't capture enough information encoded in the training data
- Low variance

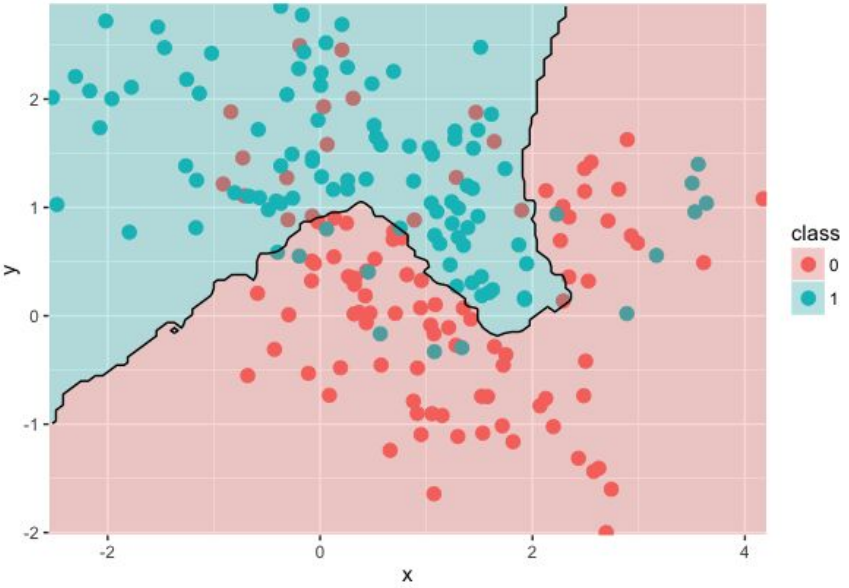


k = 101: High bias, Low variance

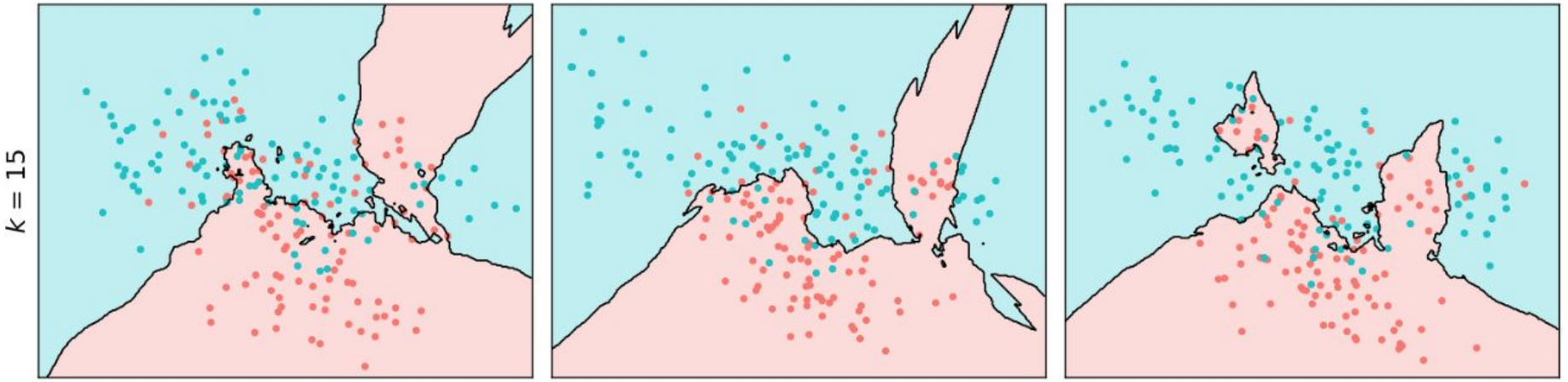


Model Selection

- Trade off bias vs. variance
- *k* = 15 “seems” just right
- This choice can/should be optimized using cross validation



k = 15: Just right

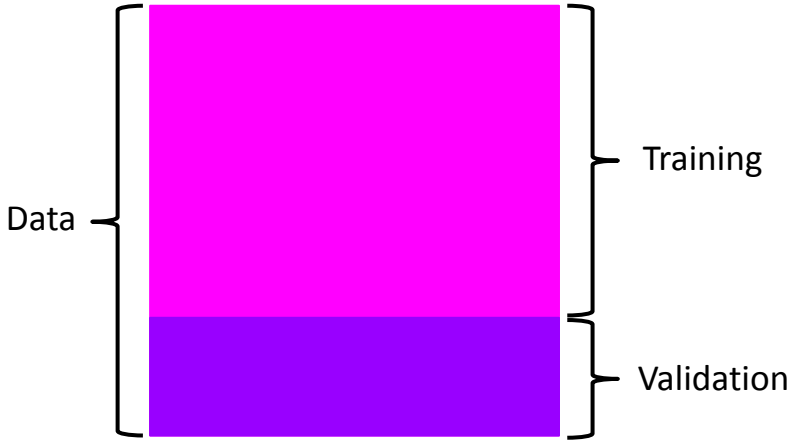


Training, Validation, and Test Data

- Divide data into (at least) two sets: training and validation sets
- As their names suggest:
  - Train your model on the training set
  - Validate your model on the validation set
  - Report (test) results on your test set
- Goal is to build a model that generalizes well from the training set (sample data) to real test (out-of-sample, in distribution) data
- To achieve this goal, use a validation set to validate your model
- Can also hold out some test data to report your final accuracy, but **cannot** retrain in light of test results

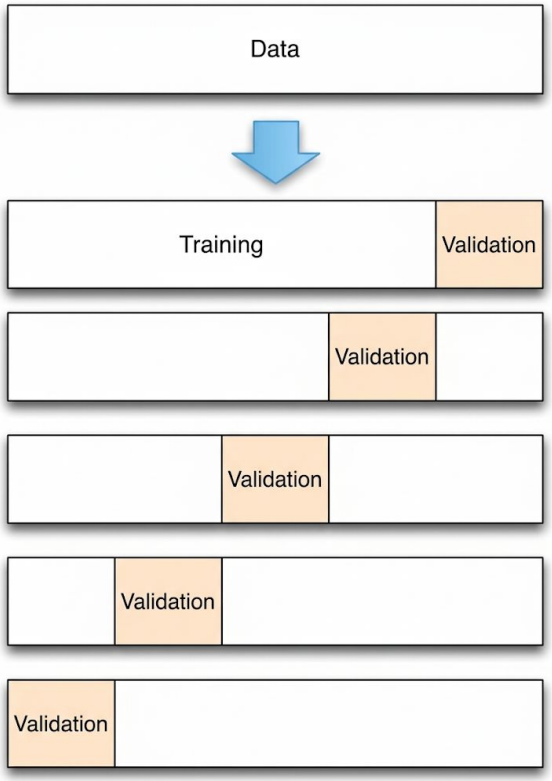
Holdout Method

- Partition training data into a large training set and smaller validation set
- Train model on the training data
- Validate model accuracy on the validation data
- Data are often shuffled first (e.g., if they were compiled by different sources)



Cross validation

- Partition data multiple times
  - If you want to partition your data 10 times, create 10 folds, and then use each fold as a test set, and the rest of the data as a training set
  - Average accuracy across all partitions to approximate model accuracy
- This is cross validation
  - Typical to use *k* partitions for *k*-fold cross validation (usually *k* = 10)
  - Leave-one-out cross validation: cross validation, to the extreme: *k* = *n*, the sample size

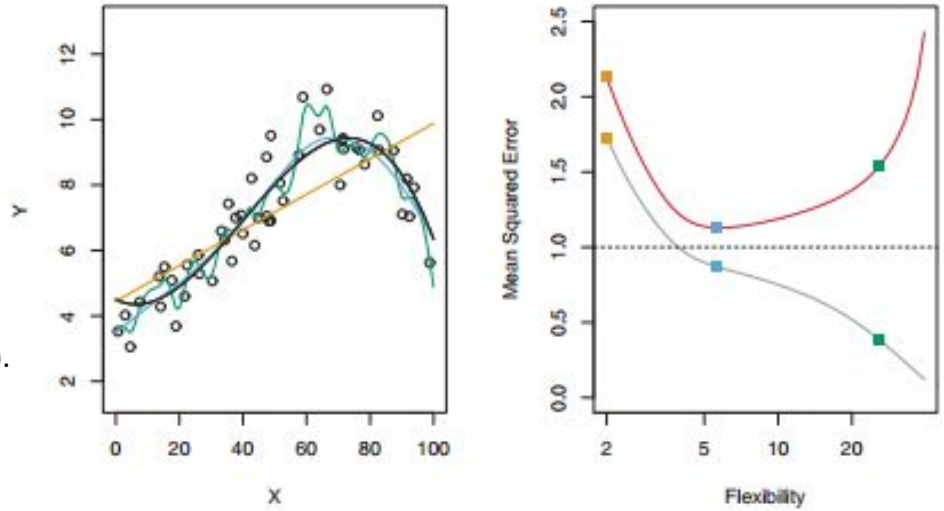


Train vs. Validation Error

The points are the data

The black line represents the true relationship

The various colors refer to different estimators (linear, quadratic, & something crazy).



The grey curve shows the training error. It decreases indefinitely.

The red curve shows the validation error. It has an **elbow**.

The colored boxes correspond to the colored fits in the left plot.

Image Source

Underfitting vs. Overfitting

- Simple (e.g., linear) models are highly biased; as such, they often underfit, meaning they fail to capture regularities in the data.
- Otoh, they are not sensitive to noise (i.e., they assume so much bias that they don't change much with the data), so are comparatively low variance.

- More complicated models are less biased. Because of their flexibility, they end up modeling noise (as well as signal), and consequently overfit.
- Flexible models have high variance, b/c the models themselves can vary enormously with the data.

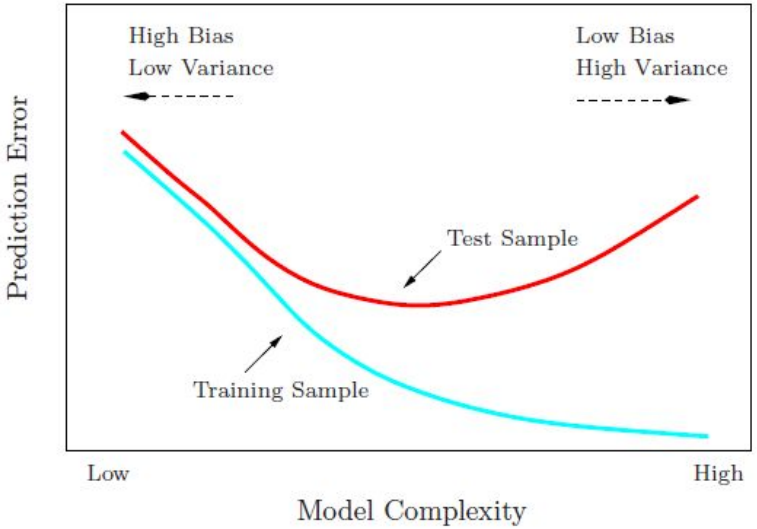


Image Source

Curse of Dimensionality

Curse of Dimensionality

Adding new features to a model (i.e., increasing the dimensionality) in the hopes of improving performance will eventually degrade performance

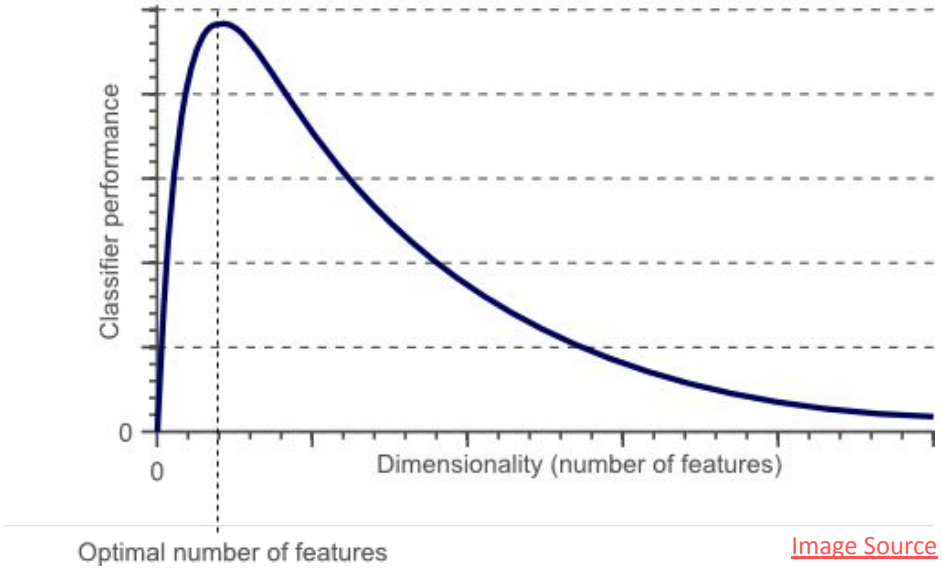


Image Source

Curse of Dimensionality

As the dimensionality of the data (i.e., the number of features) increases, Wikipedia "the volume of the space increases so fast that the available data become sparse."

Example from Stack Exchange:

- To find on your favorite kind of cookie among four possible flavors (sweet, salty, bitter, sour), requires eating four cookies.
- If there is an additional dimension, e.g., color, and there are three possible colors, you now now have to eat 4 *x* 3 = 12 cookies to find your favorite.
- Add another dimension, e.g., shape, with five possibilities, and you now have to eat 4 *x* 3 *x* 5 = 60 cookies!

Image Source

Curse of Dimensionality

As the dimensionality of the data (i.e., the number of features) increases, Wikipedia "the volume of the space increases so fast that the available data become sparse."

To cover 20% of the population:

- Need 20% of the data in 1 dimension: $(.2)^1 \sim .2$
- Need 45% of the data in 2 dimensions: $(.45)^2 \sim .2$
- Need 58% of the data in 3 dimensions: $(.58)^3 \sim .2$



